Operating Systems
Chapter 2
Operating System Overview

Instructor

Dr.Eng. Mahmoud M. Ismail

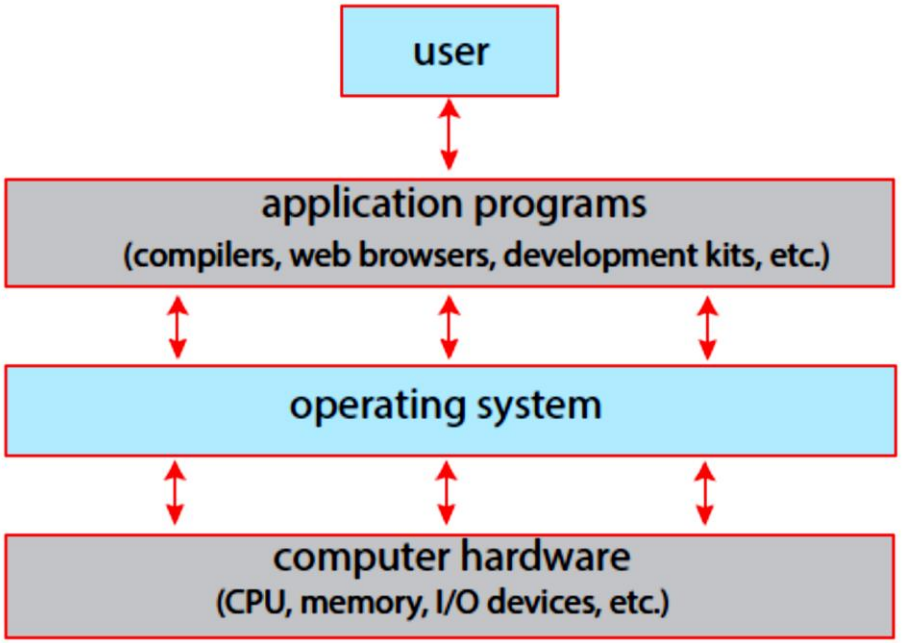
Contact: dr.eng.mahmoudmmi@gmail.com
dr.mahmoudmmi@yahoo.com

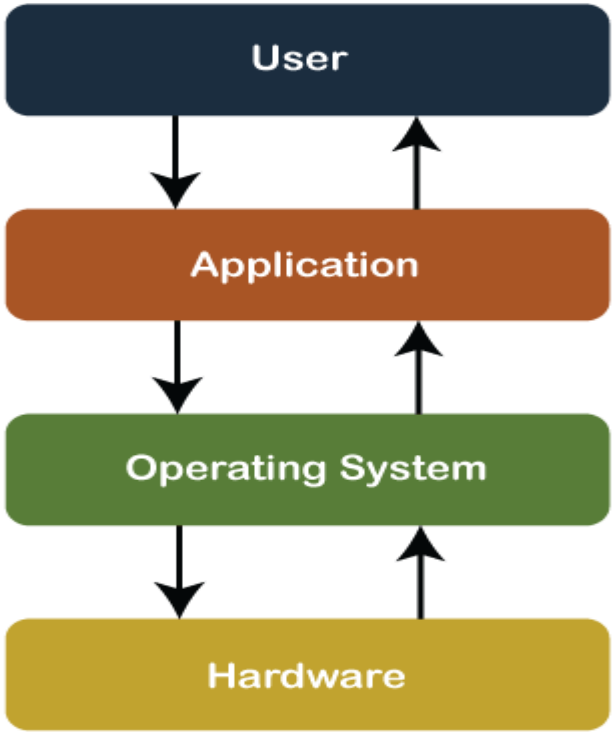
Outline

- Operating System Objectives and Functions
- Evolution of Operating Systems
- Major Achievements
- Characteristics of modern Operating Systems

Operating System

- A program that **controls** the execution of application programs
- An interface between applications and hardware





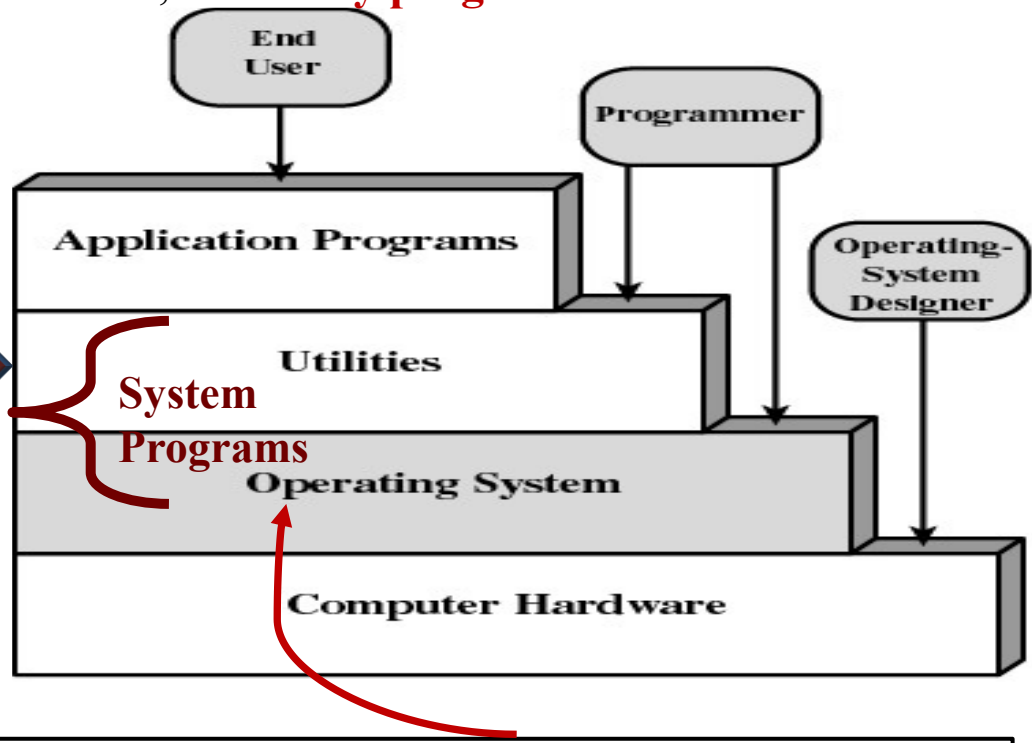
Operating System Objectives

- Convenience
  - Makes the computer more **convenient to use**
- Efficiency
  - Allows computer **resources** **(CPU time, memory space, storage space, I/O devices)** to be used in an **efficient** manner
- Ability to evolve
  - Permit effective development, testing, and introduction of **new system functions** **without** **interfering with service**.

Layers of Computer System

- The **HW** and **SW** used in **providing applications to a user** can be viewed in **a layers**
- **End user** views a computer system in terms of **a set of applications**.
- To ease **developing of an application program** by the programmer for controlling the **HW a set of system programs** is provided.
- Some of these programs are referred to as **utilities**, or **library programs**.

Frequently used functions that assist in program creation, the management of files, and the control of I/O devices



- The most important collection of **system programs** comprises the **OS**.
- The **OS** Masks **the details of the hardware** from the programmer, and provides the programmer with a convenient interface for using the system.
- It acts as **a mediator** (easier for programmer to access and use those facilities and services)

Services Provided by the OS

1) Program development

- **Editors** and **Debuggers**.
- These **services** are in the form of **utility programs** that (**not part of the core of the OS**)
- Referred to as **application program development tools**.

Services Provided by the OS

2) Program execution

Instructions and **data** must be loaded into main memory

3) Access I/O devices

Provides **an interface** so programmers access devices using simple reads and writes.

4) Controlled access to files

Provides **protection** to control access to the **files** (system with multiple users)

5) System access (For shared or public systems)

Provides **protection** of **resources** and **data** from **unauthorized users** and must **resolve** conflicts for **resource contention**.

Services Provided by the OS

6) Error detection and response

- Internal and external hardware errors
  - memory error
  - device failure
- Software errors
  - arithmetic overflow
  - division by zero
  - access forbidden memory locations
  - inability of the OS to grant the request of an application

➢ The response may
  - ending the program that caused the error
  - retrying the operation
  - **reporting** the error to the application

Services Provided by the OS

7) Accounting

- collect **statistics** for various **resources**
- monitor **performance**
  - used to anticipate **future enhancements**
  - used for **billing users** (multiuser system)

OS as a Resource Manager

- **OS is a resource manager**
  - **Manages all resources (processor execution time, main and secondary memory, I/O devices)**
    - **Allocating resources to multiple users or multiple jobs running at the same time.**
  - **Decides between conflicting requests for efficient and fair resource use (how to allocate them to specific programs and users)**

OS as a Resource Manager

- OS functions in the same way as **ordinary computer software**

  – It is **program** that is executed by **the processor**

- **The key difference** is in **the intent of the program**

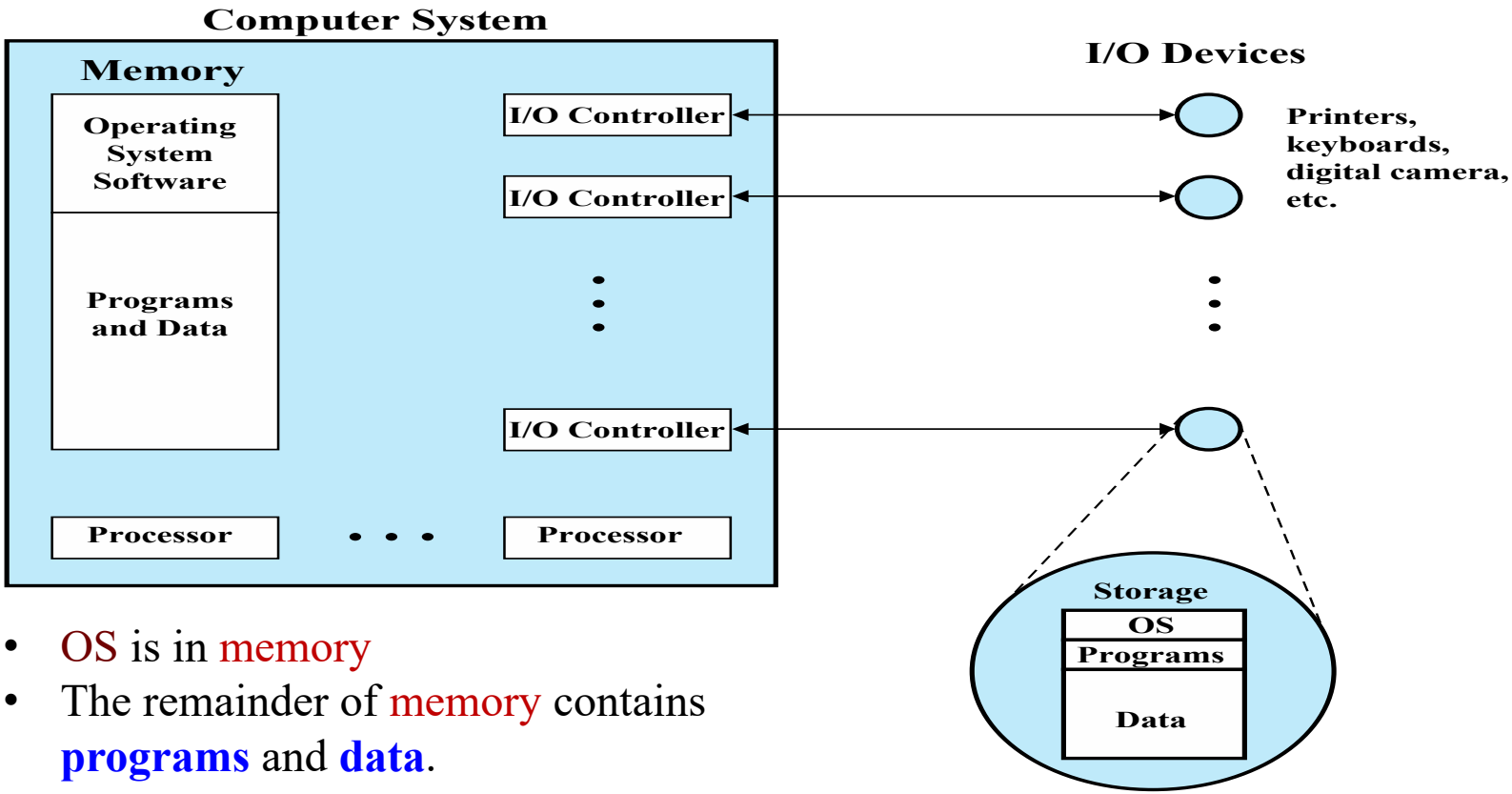
- While executing, **OS** directs **the processor**:

  1) In the **use of** other **system resources** (which **resources** are **available** for use)

  2) In the **timing of its execution of other programs** (how much processor time allocated to the execution of a particular program)

- In order for **the processor** to do any of these things, **OS relinquishes control of** the processor **to execute other programs**.

- Then, **resumes control** to prepare **the processor** to do the next piece of work.



- OS is in memory
- The remainder of memory contains **programs** and **data**.

- OS **decides** when an I/O device can be used by a program in execution.
- **Processor** itself is **a resource**, and the OS must **determine** how much **processor time** is to be devoted to the **execution of a particular user program**.

The Operating System as Resource Manager

Kernel

- Portion of OS that is in main memory
- Contains **most-frequently used functions**
- Also called the nucleus

Evolution of an Operating System

- OS will evolve for a number of reasons:

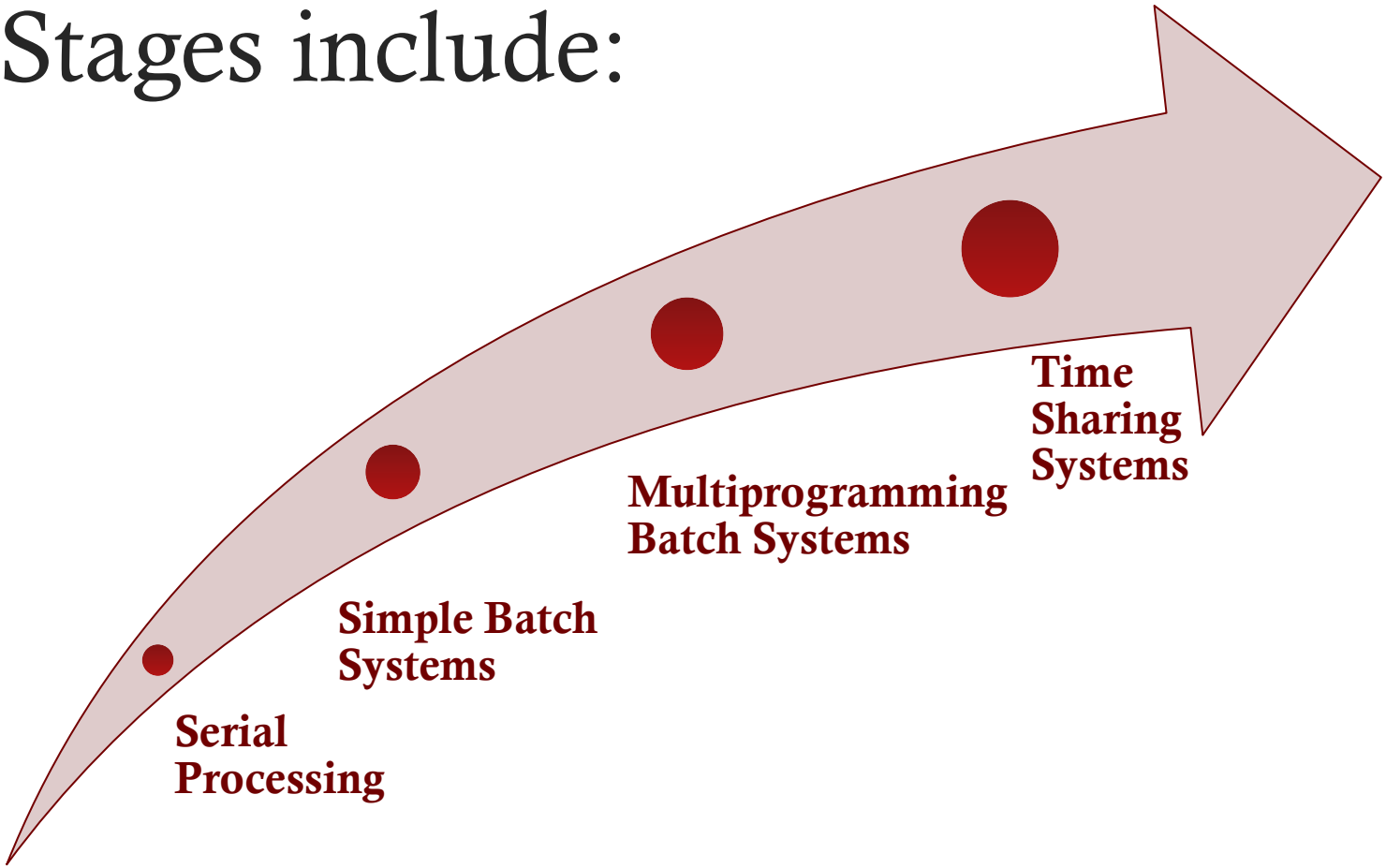
1) **Hardware upgrades and new types of hardware**
   - **For example, early versions** of UNIX and the Macintosh OS did not employ **a paging mechanism** because they were run on processors without **paging hardware**.
   - Subsequent versions of these OS were **modified to exploit paging capabilities**

2) **New services** (new tools and new applications)
   - **For example,** if it is found to be difficult to **maintain** good performance for users with existing tools, new measurement and control tools may be added to the OS.

2) **Fixes** (faults are discovered and fixes are made)

Evolution of Operating Systems

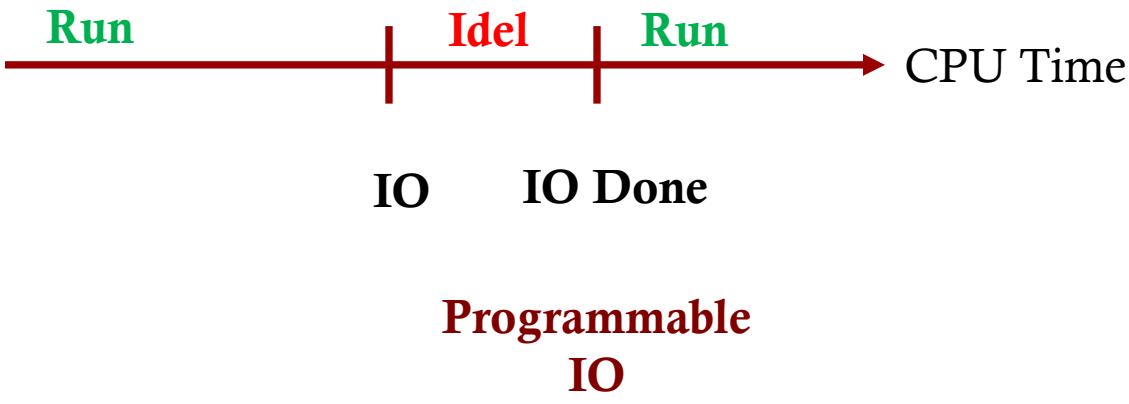
- Stages include:



Serial Processing (one by one no interleaving)

Earliest Computers:

- Size of computer is large
- User has no direct communication with computer.
- A person called administrator receives a code from user in the form punched cards by device which convert the written code into holes on cards.
- Programs were loaded via the input device (card reader).
- Admin determines which program will run (human scheduling)



Users have access to the computer in "**series**"

Serial Processing Problems

1) Scheduling

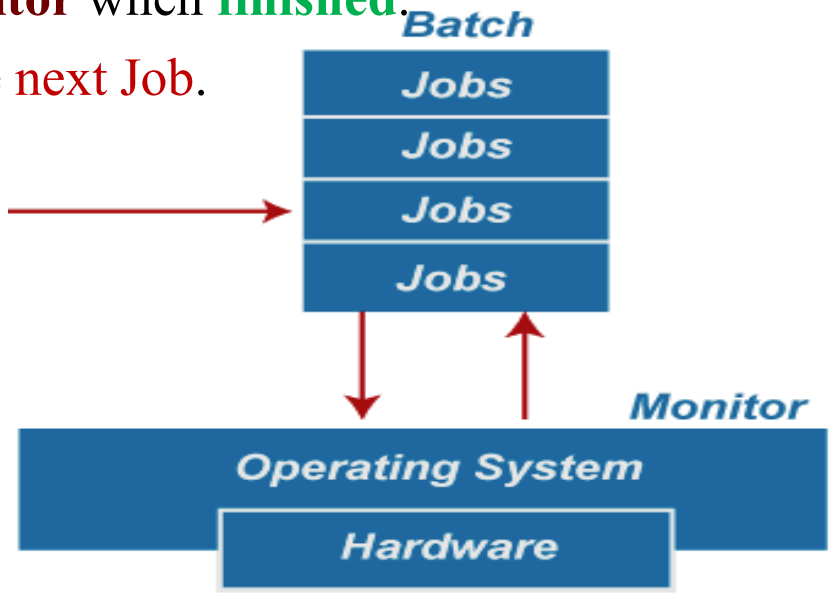
- Sign-up sheet to reserve **processor time**
  - Time allocations could run **short** or **long**, resulting in **wasted computer processing time**.
  - If problems occurred the user could be forced to stop before resolving the problem

2) Setup time

- A considerable amount of **time was spent on setting up the program to run**:
  - **Loading the compiler** plus the high-level language program (**source program**) into memory.
  - **Saving the compiled program** (object program)
  - **Loading and linking together the object program** and **common functions**.

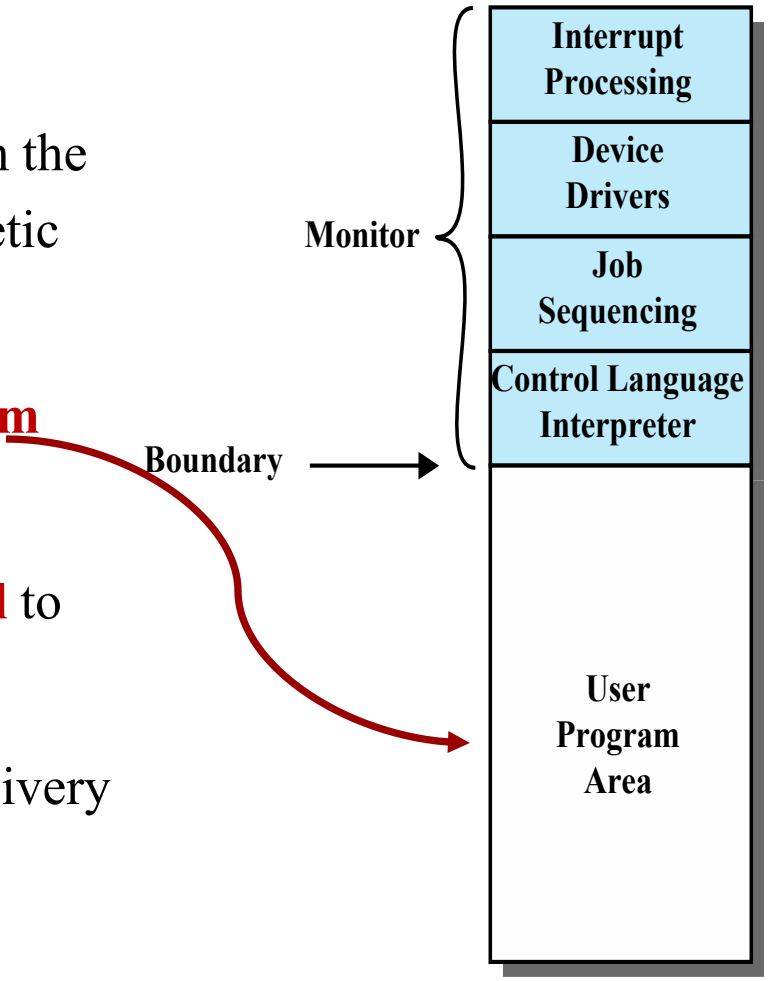
Simple Batch Systems
Uniprogramming (Single Program)
(Offline No Interactivity)

- Use **software** known as the **monitor** in **memory** that **controls the running programs**
  - The user **submits** the job on cards or tape to a computer **operator**
  - Jobs are **batched** together by type of **languages** and requirement in queue.
  - **Operator** places **a batch of several jobs** on an input device for use by **monitor**.
  - **Monitor**, **manages** the execution of **each job** in the **batch**
  - **Program** constructed to **branch back** to **monitor** when **finished**.
  - The **monitor** automatically begins **loading** the next Job.



Simple Batch Systems

- The **monitor reads** in **jobs** one at a time from the input device (typically a card reader or magnetic tape drive).
- The **current job** is placed in the **user program area**, and control is passed to this job.
- When the **job** is **completed**, it returns **control** to the **monitor**, which reads in the **next job**.
- The results of **each job** are printed out for delivery to the user.



Memory Layout for Resident Monitor

Advantages of Batch OS

1) **Monitor handles the scheduling problem**

- **A batch of jobs is queued up, and are executed as rapidly as possible, with no idle time. (eliminates CPU time between two jobs)**

2) **The monitor improves job setup time :Job Control Language (JCL)**

- With **each job**, **instructions** are included in a job control language
- This is **a special type of programming language** used to **provide** instructions to the **monitor**.
  - what compiler to use
  - Loading object programs into memory
  - what data to use

Example

- User submitting a program written in ***FORTRAN*** plus some ***data***.
- All ***FORTRAN*** instructions and ***data*** are on a separate punched card or a separate record on tape.
- **Job** includes job control instructions, which are denoted by the beginning "**$**".

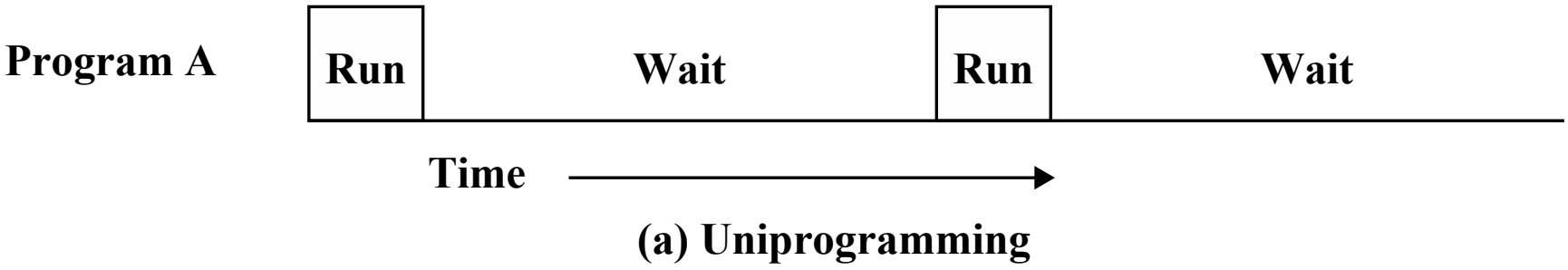
```
$JOB
$FTN              load the FORTRAN compiler
...       Fortran instructions
$LOAD             loads the object code (in place of compiler)

$RUN              run the program
...       Data
$END
```

Disadvantage of Batch OS

Uniprogramming (single program)

- Processor must wait for I/O instruction to complete before preceding



(a) Uniprogramming

Desirable Hardware Features

Memory protection for monitor

- Do not allow the memory area containing the monitor to be altered

Timer

- Prevents **a job** from **monopolizing the system**
  - The **timer** is set at **the beginning** of **each job**.
  - If the **timer** expires, **program** is **stopped**, and **control** returns to **the monitor**.

Problems with Simple Batch Systems

- Processor is often idle (I/O devices slow compared to processor)
  - Consider Program that processes a file of records

Read one record from file	0.0015 sec
Execute 100 instructions	0.0001 sec
Write one record to file	0.0015 sec
	Total 0.0031 sec

CPU utilization

(Total Time Spent on Non Idle Tasks / Total Time) x 100

0.0001/0.0031 = 3.2%

In this example, the computer spends over **96%** of its time **waiting** for I/O devices to finish transferring data to and from the file

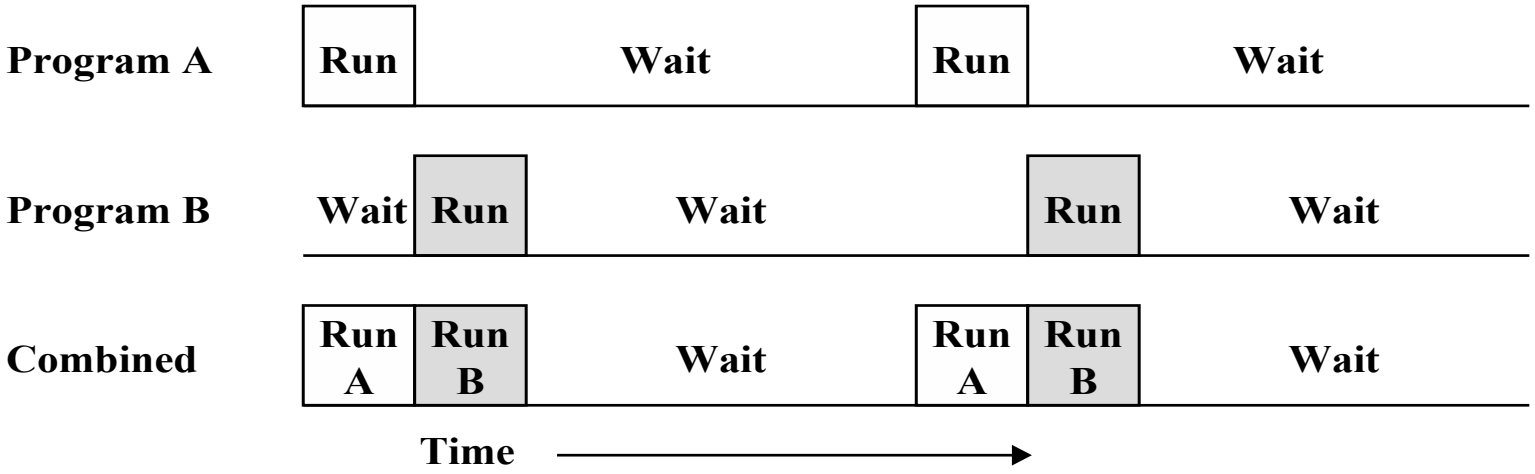
Multiprogramming Batch Systems

- When one job needs I/O, the processor can switch to the other job, which is likely not waiting for I/O → Maximize Throughput.



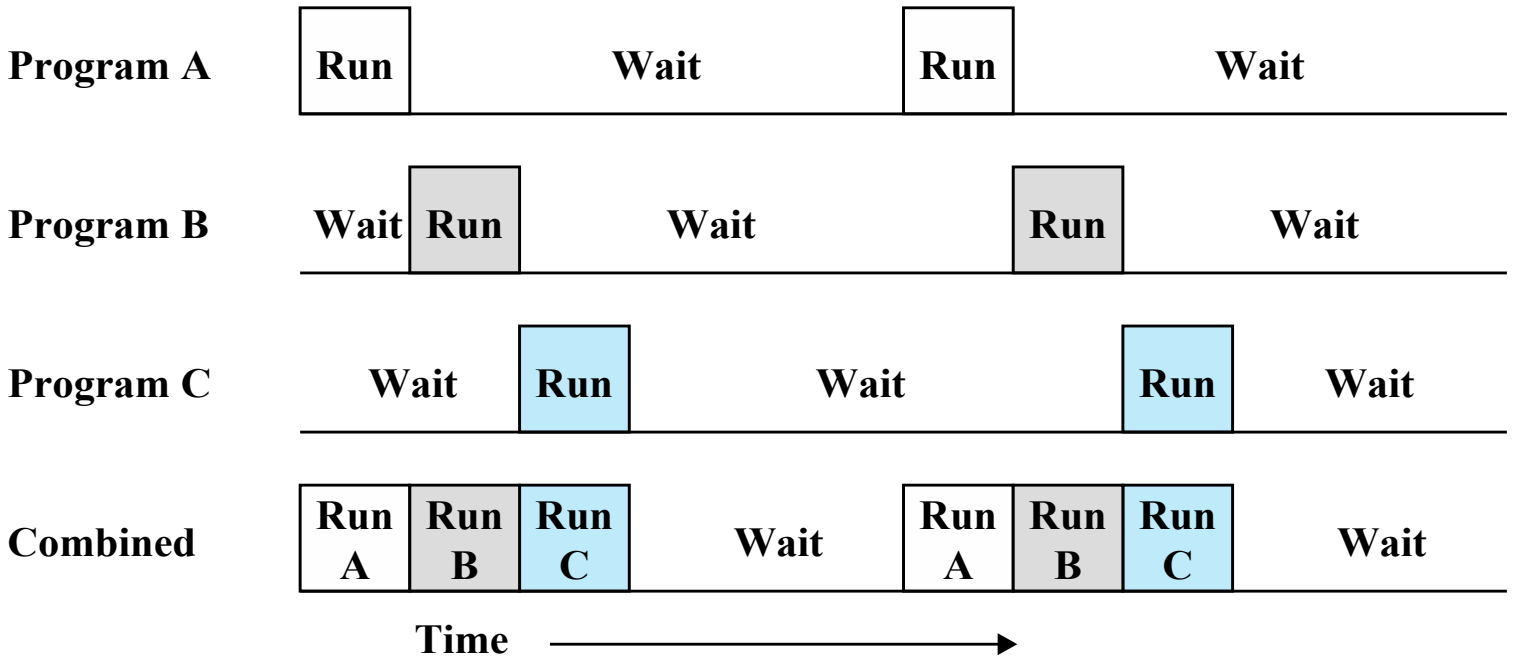
Interrupt Driven
DMA

Multiprogramming Batch Systems



Multiprogramming with two programs

Multiprogramming with three programs



- **Memory** is expanded to hold three, four, or more programs and **switch** among all of them.

Requirements for Multiprogramming

Memory management

- To have **several jobs ready to run**, they must be kept in main memory, requiring some form of **memory management**.

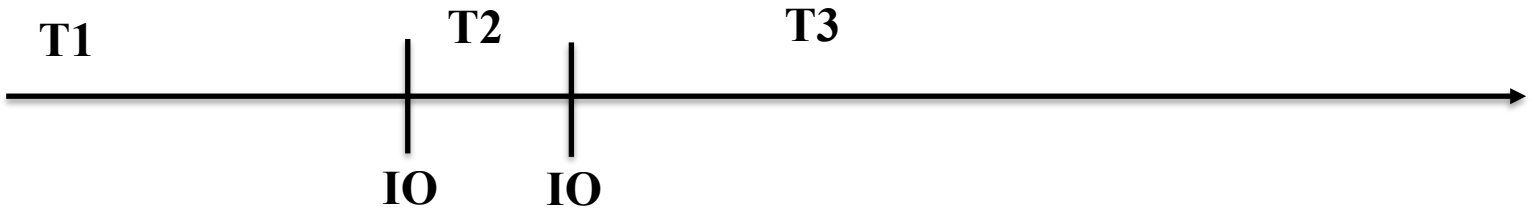
Scheduling

- If **several jobs** are **ready to run**, **the processor** must decide **which one to run**, and this decision requires an **algorithm for scheduling**.

Time-Sharing Operating System

Objective of MP is to increase Throughput but Objective of TS is to reduce response time witch leads to Interactivity.

In MP



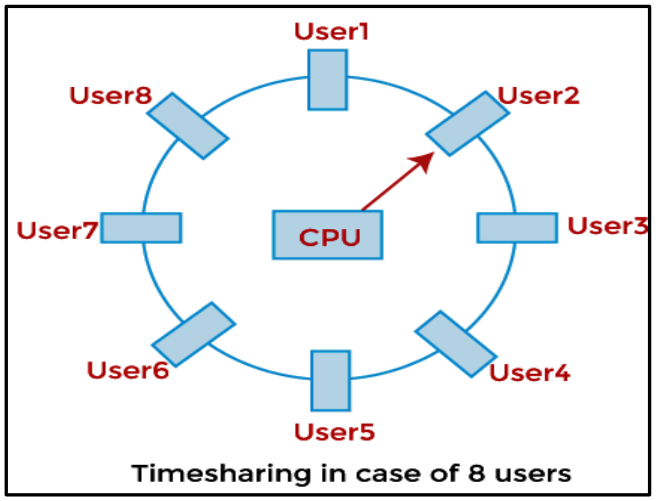
If T1 did not request IO → monopolize processor

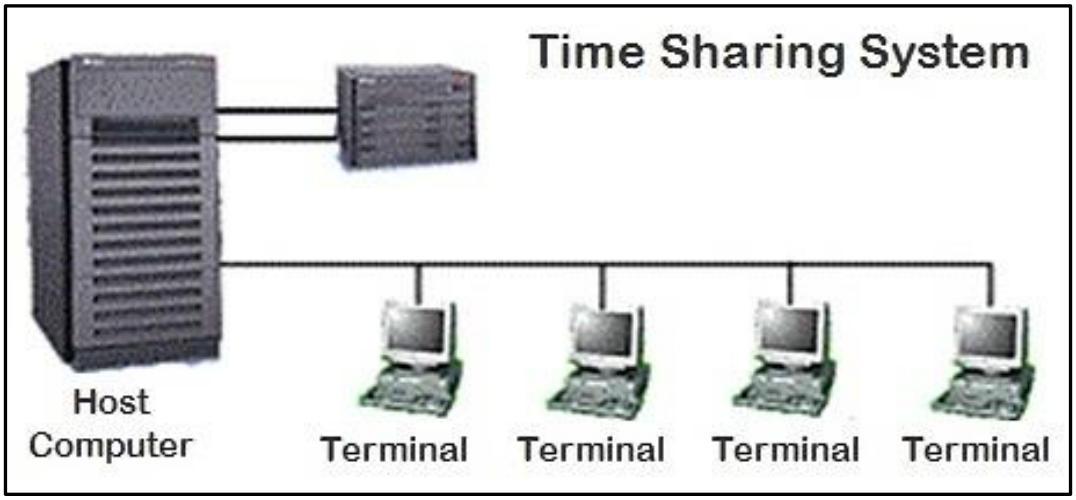


Time-Sharing Operating System

Solution

- **Processor's time** is **shared** among **multiple users**
- OS ***interleaving*** execution of each user program (**alternating between them mutually**) in a short burst or ***quantum***.





❖ **Multiple users** simultaneously **access** the system through **terminals**

Time-Sharing Operating System

Solution

Max Processor Time (*Quantum*/*Time Slice*) for all Tasks



Each Task will have the chance to run after certain time (Q) → reduce response time witch leads to Interactivity.

Time-Sharing Operating System

How Q is calculated (is Q for processor 3 Ghz is the same as processor 2.2 Ghz)?

- **Q (time) is function of Cycle Time**

Cycle Time (CT) is the minimum time to execute one instruction =1/Freq.

- **Assume processor 3 Ghz**

Cycle Time (CT) =1/3 * 10^9 =0.333 * 10^{-9} sec (0.333 nano sec)

- **If Q is set to CT, all tasks will execute only one instruction → overhead**

✓ **So, Q = multiples of Cycle Time = say 10 Cycle Time**

- **How Cycle Time is determined?**

OS Query BOIS for the driver of processor to get GHZ of processor.

➢ Too short quantum causes:

Many process context switches → **overhead** and reduces CPU efficiency.

➢ Too long quantum causes:

1) The other processes in the ready queue must wait longer between turns
2) poor response to short interactive processes.

➢ Setting Q to INFINITY → No Interactivity

How do we know Q expired ???

Another timer settled for Q

Clock interrupt means cycle time done but not equal Q

Can we able to increase Throughput and reduce response time at the same time???

Not all cases

- **Some cases both are contradictive**
- **Special case may achieve both according to tasks**

Context Switching



PC: Addresses instructions of T1

When Q expired

PC: Addresses instructions of T2

- **How returning to T1 ???**

✓ **The PC value of T1 must be saved when switching to T2. (Context Switching)**

1) **Save state T1 (Processor registers in main memory)**
2) **OS decides which will run next (Processor scheduling) T2 will run**
3) **Load state T2 into Processor registers from main memory**

Batch Multiprogramming vs Time Sharing

	Batch Multiprogramming	Time Sharing
Principal objective	Maximize Throughput (processor use) ➢ Multiprogramming makes sure that the CPU always has something to execute, thus increases the CPU utilization.	Minimize response time ➢ User requests can make in small-time responses.
Source of directives to operating system	Job control language commands provided with the job	Commands entered at the terminal

Major Achievements

Theoretical advances in development of OS

1) **Processes Ch.3**
2) **Memory management Ch.6**
3) **Information protection and security**
4) **Scheduling and resource management Ch.8**

Processes

- A program in execution
- An instance of a program running on a computer
- The entity that can be assigned to and executed on a processor
- A unit of activity characterized by a single sequential thread of execution, a current state, and an associated set of system resources

Factors contributing to development of process concept

Multiprogramming batch operation

- **Processor is switched among the programs residing in main memory**

Time sharing

- **Be responsive to the individual user but be able to support many users simultaneously**

Real-time transaction systems

- **A number of users are entering queries or updates against a database**

Difficulties with Designing System Software

1) Improper synchronization
   - Ensure a process waiting for an **I/O device** receives the signal
     - e.g. a program that initiates an **I/O must wait until the data** are **available** in a buffer before proceeding.
     - In such cases, a signal is required.
     - Improper design of the signaling mechanism can result in signals being lost or duplicate signals being received.

Difficulties with Designing System Software

2) Failed mutual exclusion

- More than one user or program will attempt to make use of **a shared resource** at the same time.

– e.g. two users may attempt to **edit the same file** at the same time.

- There must be some sort of mutual exclusion mechanism that permits only one program at a time to perform an **update** against **the file**.

3) Nondeterminate program operation

- The results of a particular program normally should depend only on the input to that program and not on the activities of other programs in a shared system.
- When programs share memory, and their execution is interleaved (alternating between them) by the processor, they may interfere with each other by overwriting common memory areas.
- Thus, **the order** in which programs are scheduled may affect the outcome of any particular program.

4) Deadlocks

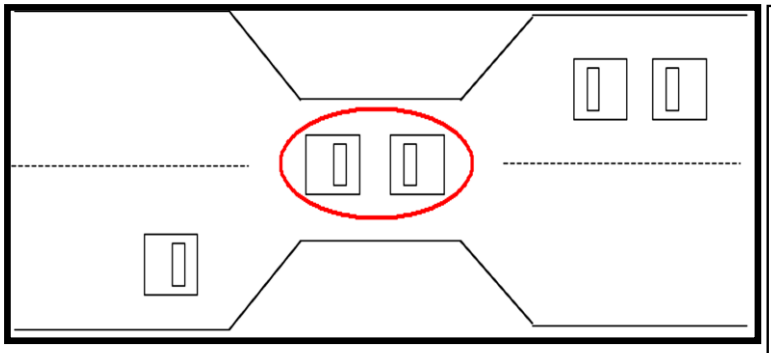
- It is possible for two or more programs to be hung up waiting for each other.

➢ e.g. two programs may each require two I/O devices to perform some operation (e.g., disk to tape copy).

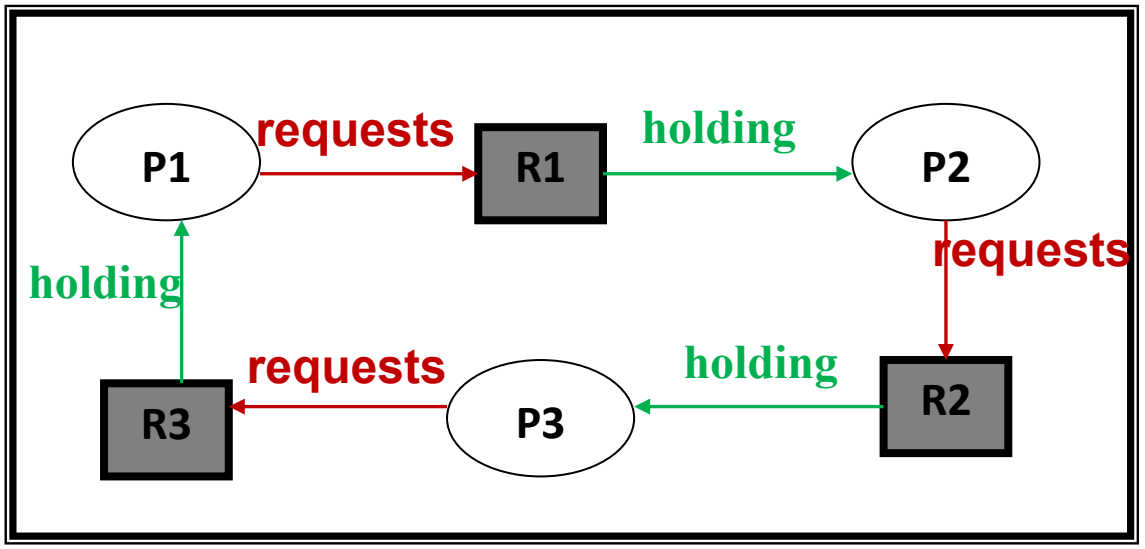
– One of the programs has seized control of one of the devices and the other program has control of the other device.

– Each is **waiting** for the other program to release the desired resource.

- Such a deadlock may depend on **the timing of** resource allocation **and release**.



Analogous example



Components of Process

- Consists of three components

1) An executable program
2) Associated data needed by the program
3) Execution context of the program
   - All information the operating system needs to manage the process (process state)
     - Contents of various processor registers such as PC
     - Process priority
     - Whether the process is waiting for the completion of an **I/O event**

Processes management

- Two processes, A and B exist in portions of memory.
- Each process is recorded in a **process list** built and maintained by the OS.
- The process list contains one entry for each process, which includes a pointer to the location of the block of memory that contains the process.
- The process **index register** contains the index into the process list of the process currently controlling the processor.
- The **PC** points to the next instruction in that process to be executed.
- The **base** and **limit registers** define the region in memory occupied by the process.
- The **base register** is the starting address of the region of memory and **the limit** is the size of the region (in bytes or words).
- The PC and all data references are interpreted relative to **the base register** and must not exceed the value in **the limit register**.
- This prevents interprocess interference.

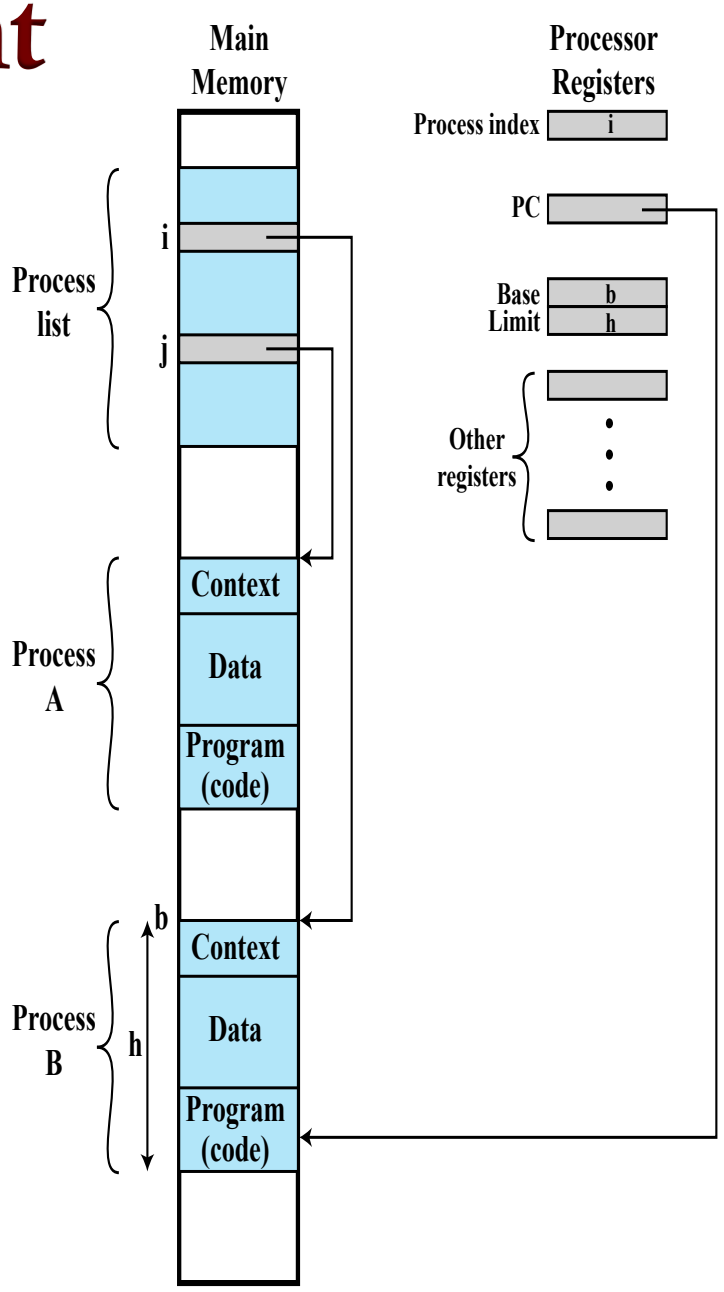


Figure 2.8 Typical Process Implementation

- The process **index register** indicates that process B is executing.
- Process A was previously executing **but** has been temporarily interrupted.
- The contents of all the registers at the moment of A's interruption were recorded in its execution context.
- Later, the OS can perform a process switch and resume execution of process A.
  - The process switch consists of storing the context of B and restoring the context of A.
- When the PC is loaded with a value pointing into A's program area, process A will automatically resume execution.

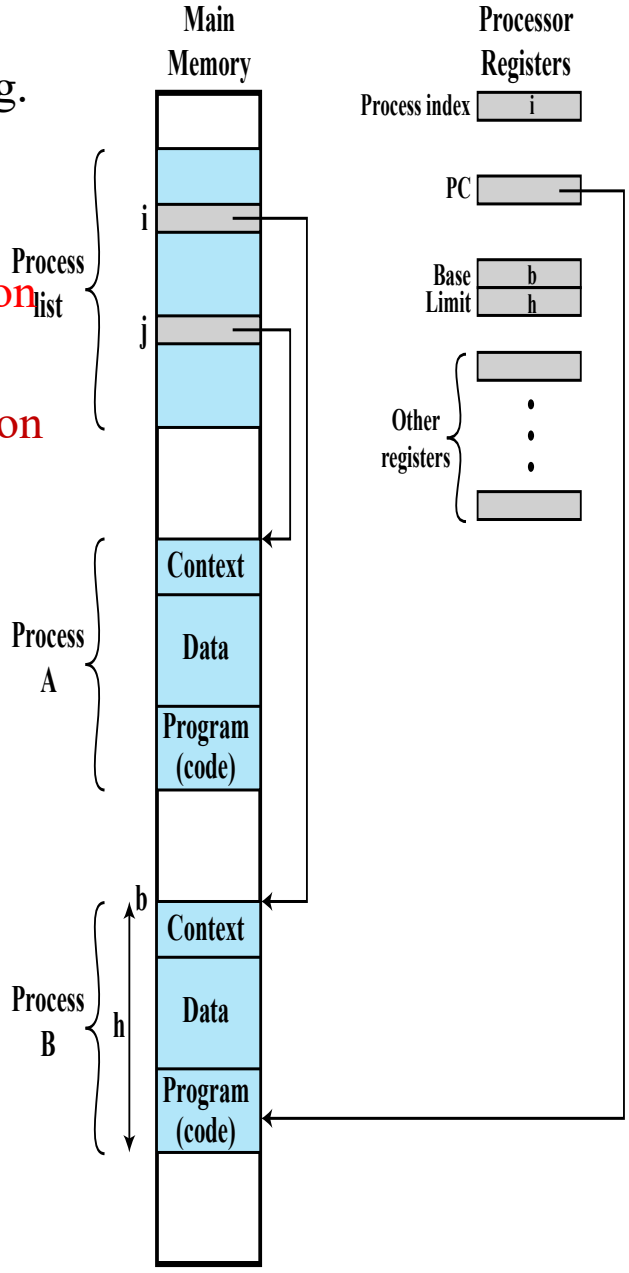


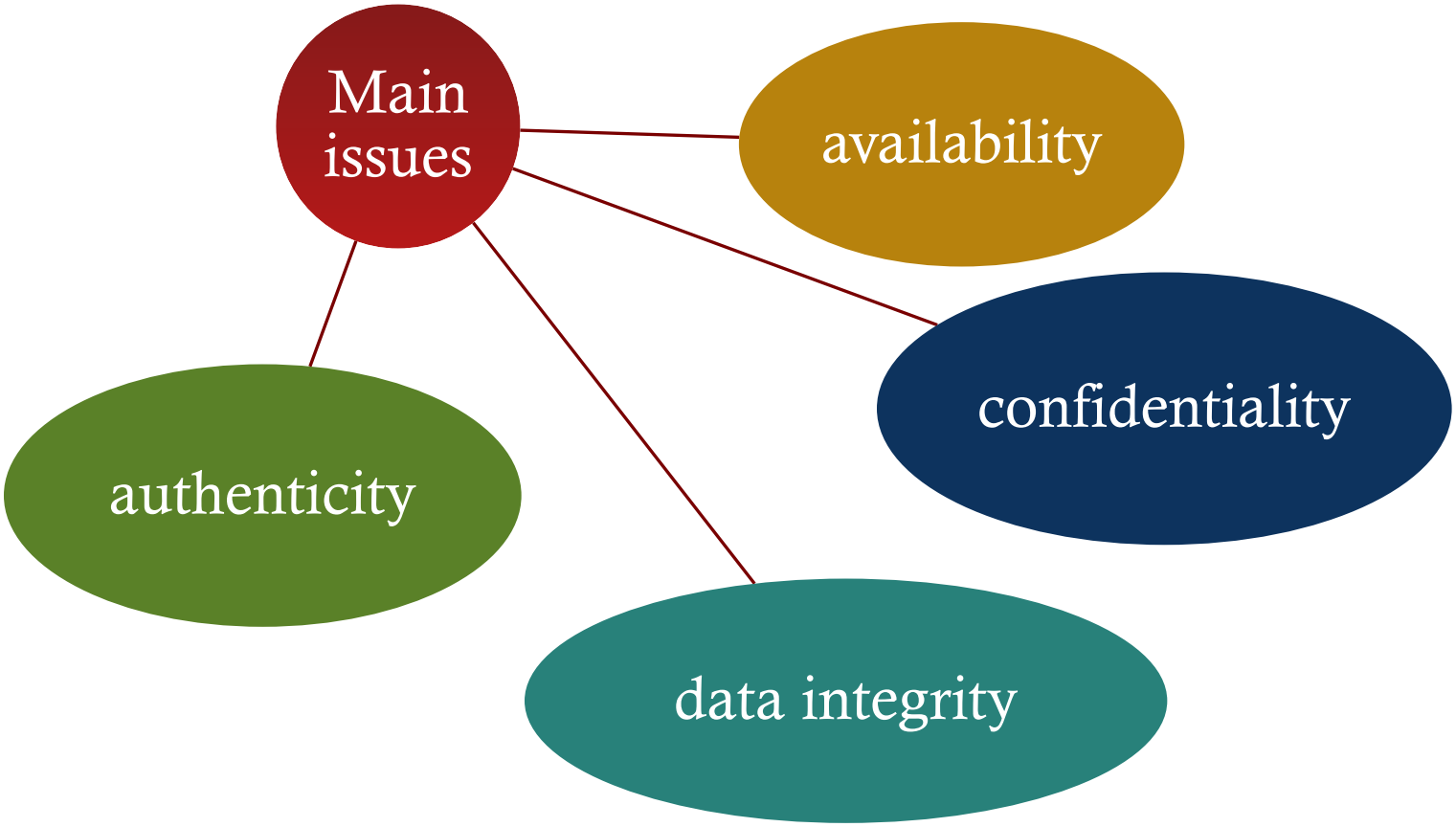
Figure 2.8 Typical Process Implementation

Information Protection and Security

- Access control
  - Regulate user access to the system
- Information flow control
  - Regulate flow of data within the system and its delivery **to** users
- Certification
  - Proving that access and flow control perform **according to** specifications.

Information Protection and Security

Security and protection as it relates to operating systems can be roughly grouped into four categories:



Information Protection and Security

- **Availability**
  - Protecting the system against **interruption**
- **Confidentiality** (**access control**)
  - Assuring that **users cannot read data for which access is unauthorized**
- **Data integrity** (**access control**)
  - Protection of data from **unauthorized modification**
- **Authenticity** (**login**)
  - Proper verification of the **identity of users** and the validity of messages or data

Scheduling and Resource Management

- Fairness
  - give equal and fair access of resource to all processes
- Differential responsiveness
  - discriminate between **different classes of jobs** with different service requirements.
- Efficiency
  - maximize throughput, minimize response time, and accommodate as many users as possible (time sharing).

Major elements of the OS involved in the scheduling of processes and the allocation of resources in a multiprogramming environment

1) Processor Scheduler

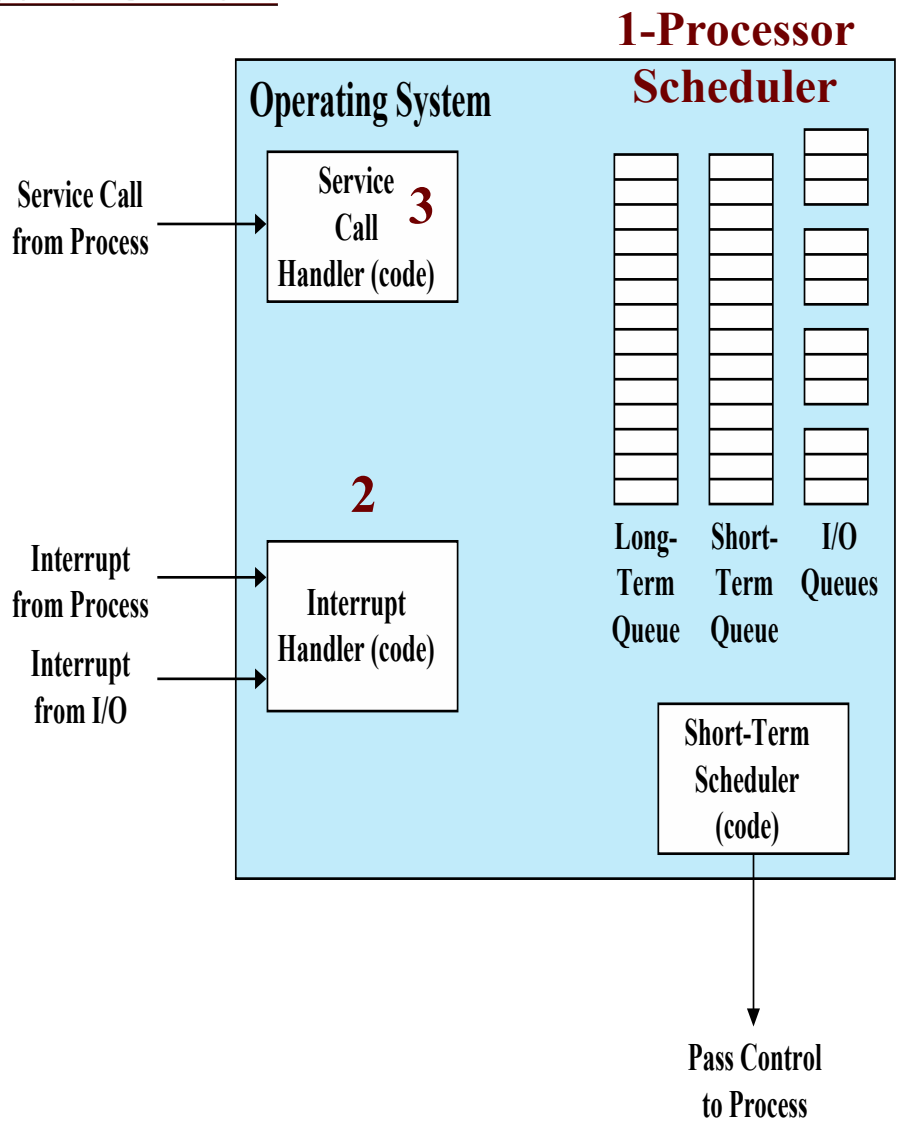
- The OS maintains a number of queues, each of which is simply a list of processes waiting for some resource.

a) **short-term queue** consists of **processes** that are in main memory and are ready to run.

- It is up to the short-term scheduler (dispatcher) to pick one.

– A common strategy is to give each process in the queue **some time in turn**; this is referred to as **a round-robin technique** (circular queue).

– Another strategy is to assign **priority** levels to the various processes.



Key Elements of an Operating System for Multiprogramming

b) **The long-term queue** is a list of **new jobs** waiting to use the **processor**.

- The OS adds **jobs** to system by **transferring a process** from the **long-term queue** to the **short-term queue**.

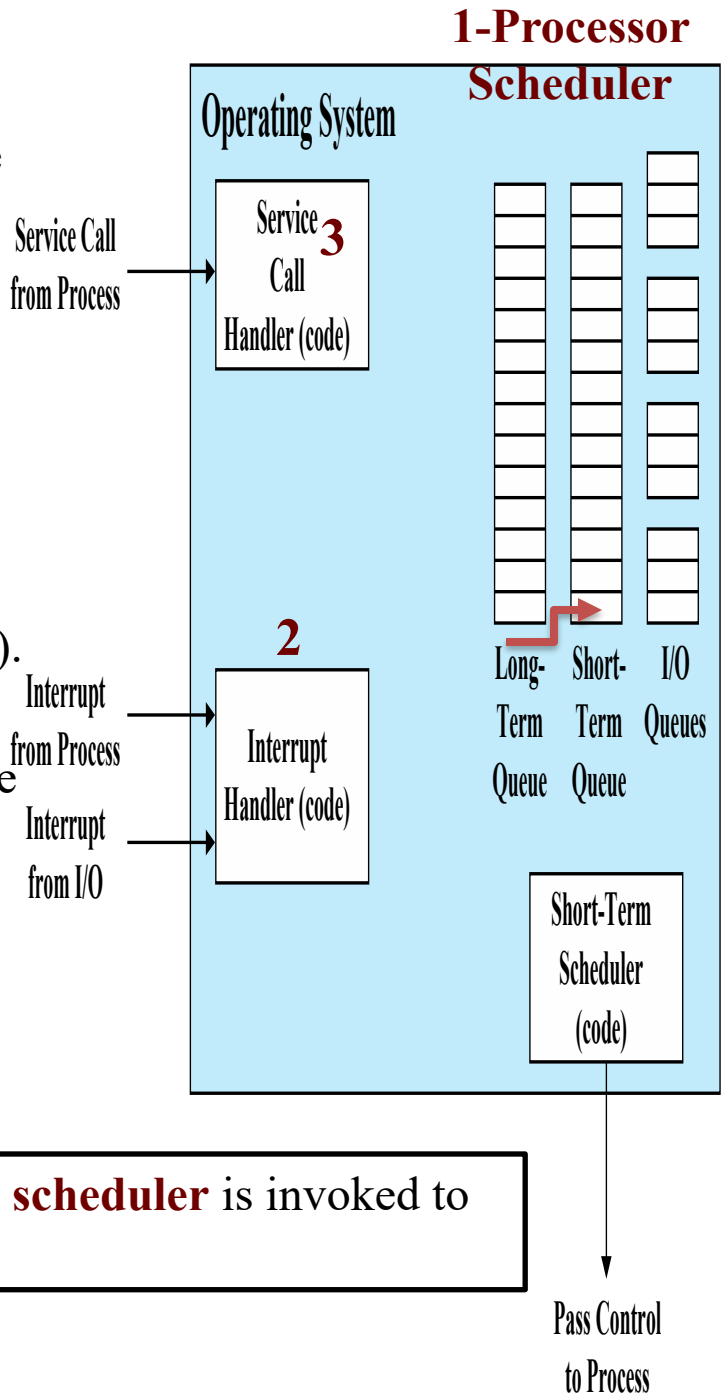
c) **I/O queue** for each **I/O device**.

- **All processes** waiting to use **each device** are lined up in that **device's queue**.

2) **The interrupt handler** if an interrupt occurs (request IO).

3) **Service/system call handler** from a process

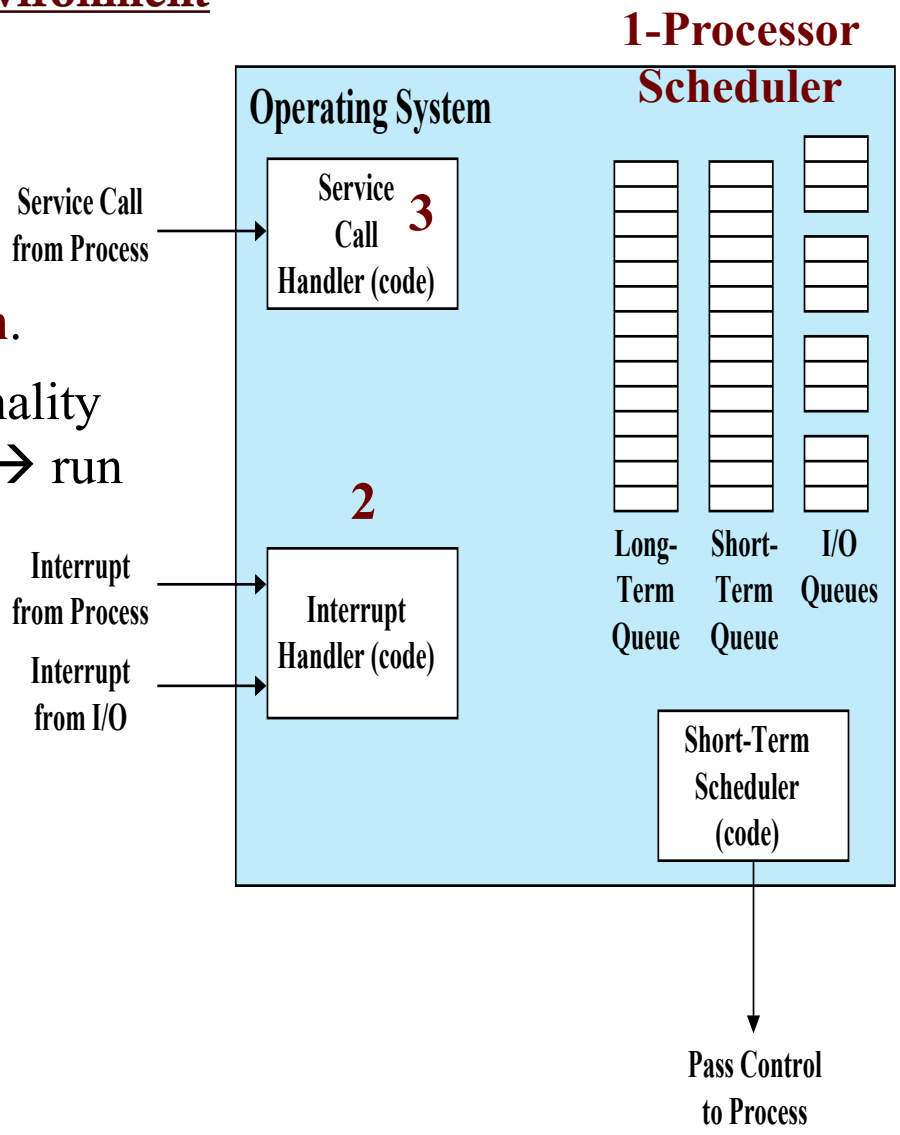
- A process may invoke OS service, such as an I/O device handler, by means of **a service call**.



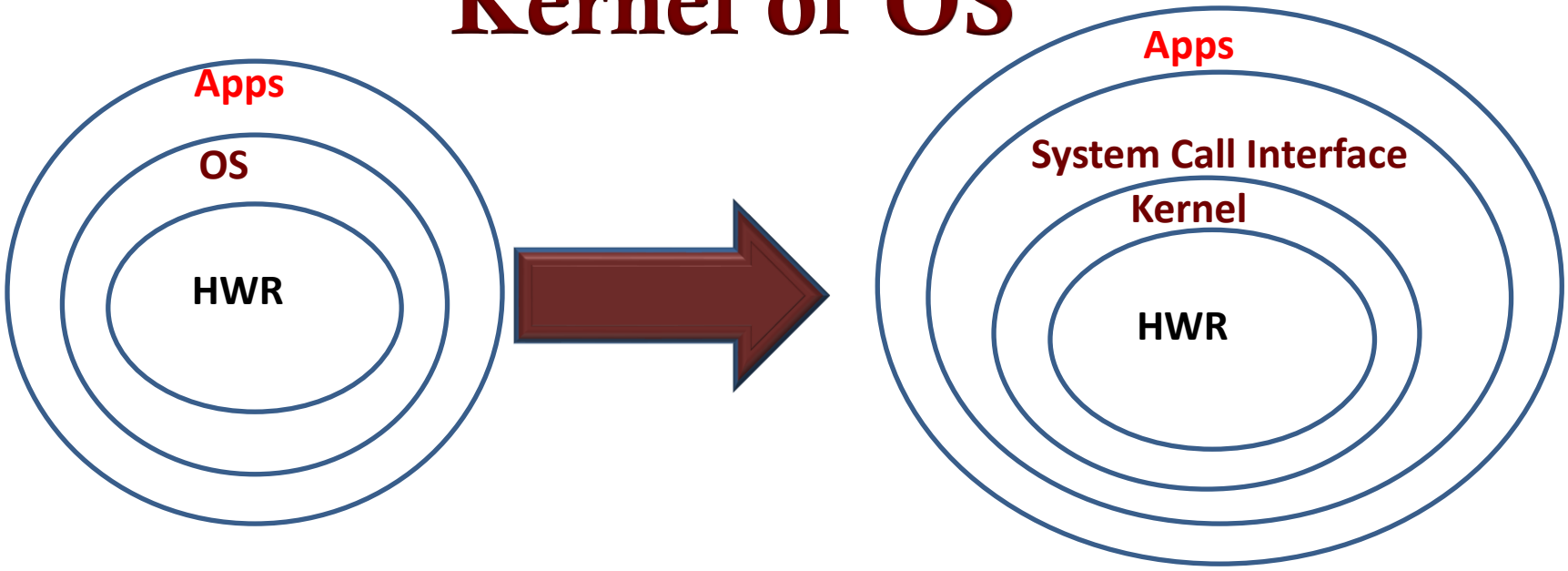
Once the interrupt or service call is handled, **the short-term scheduler** is invoked to pick a process for execution.

Major elements of the OS involved in the scheduling of processes and the allocation of resources in a time sharing environment

- **The Scheduler** must consider **the quantum**.
- **The interrupt handler** is the same functionality **plus** interrupt when **the quantum expired** → run a new process.
- **Service call handler** is the same



Kernel of OS



- **Kernel is the main part of OS that must be available in Main Memory.**
- **One of the developments leading to modern OS**
- **Most OS featured a large OS functionalities are provided in this kernel including :**

1) **Processor Scheduling**
2) **Interprocess communication**
   - **(Pipe and socket is a data structure in kernel implemented on the level of OS for communication between processes)**
3) **Process management**
4) **Memory management**
5) **Secondary storage management**
6) **I/O devices management**

Essential functionalities

Characteristics of Modern Operating Systems

1) **Microkernel architecture**
2) **Multithreading (Ch.4)**
3) **Multicore (chip multiprocessor)**
4) **Clustered Systems**
5) **Multiprocessor Symmetric Multiprocessors (SMP)**
6) **Distributed operating systems**
7) **Object-oriented design**

Characteristics of Modern Operating Systems

Microkernel architecture

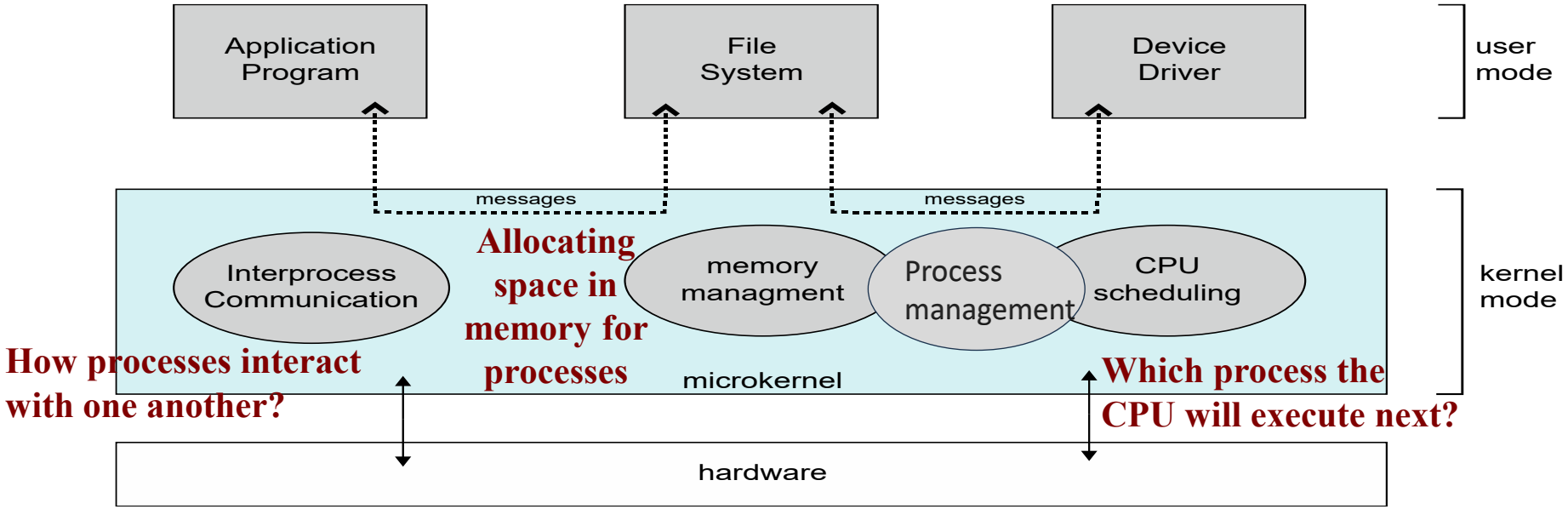
- Assigns only essential functions to the kernel (1 to 4)
- Other OS services 5 and 6 are part of OS but not loaded at first in memory (On demand) and provided by processes (called servers) that run in user mode.
- Servers may be customized to specific application or environment requirements
  - ✓ When the machine has Keyboard → load the part that manage KB
  - ✓ When the machine has network card → load the part that manage it

Microkernel architecture

– **Microkernel structures OS by removing all nonessential components from kernel and implemented them as system processes at user processes level.**

– **System processes will have privilege higher than user processes (because they part of OS)**

e.g. CTRL+ALT+DEL → user processes + system processes (part of OS loaded in memory but not part of Microkernel)

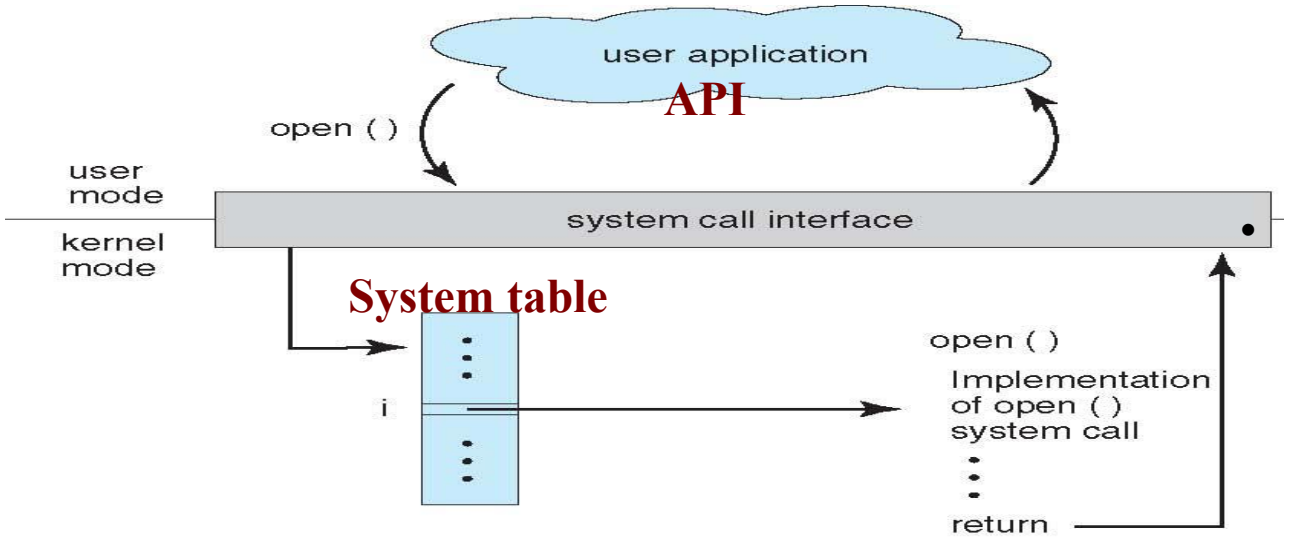


➢ Decouple **kernel** and **server** development

–When the **kernel** needed to be modified, the modification will be **fewer**

System Calls

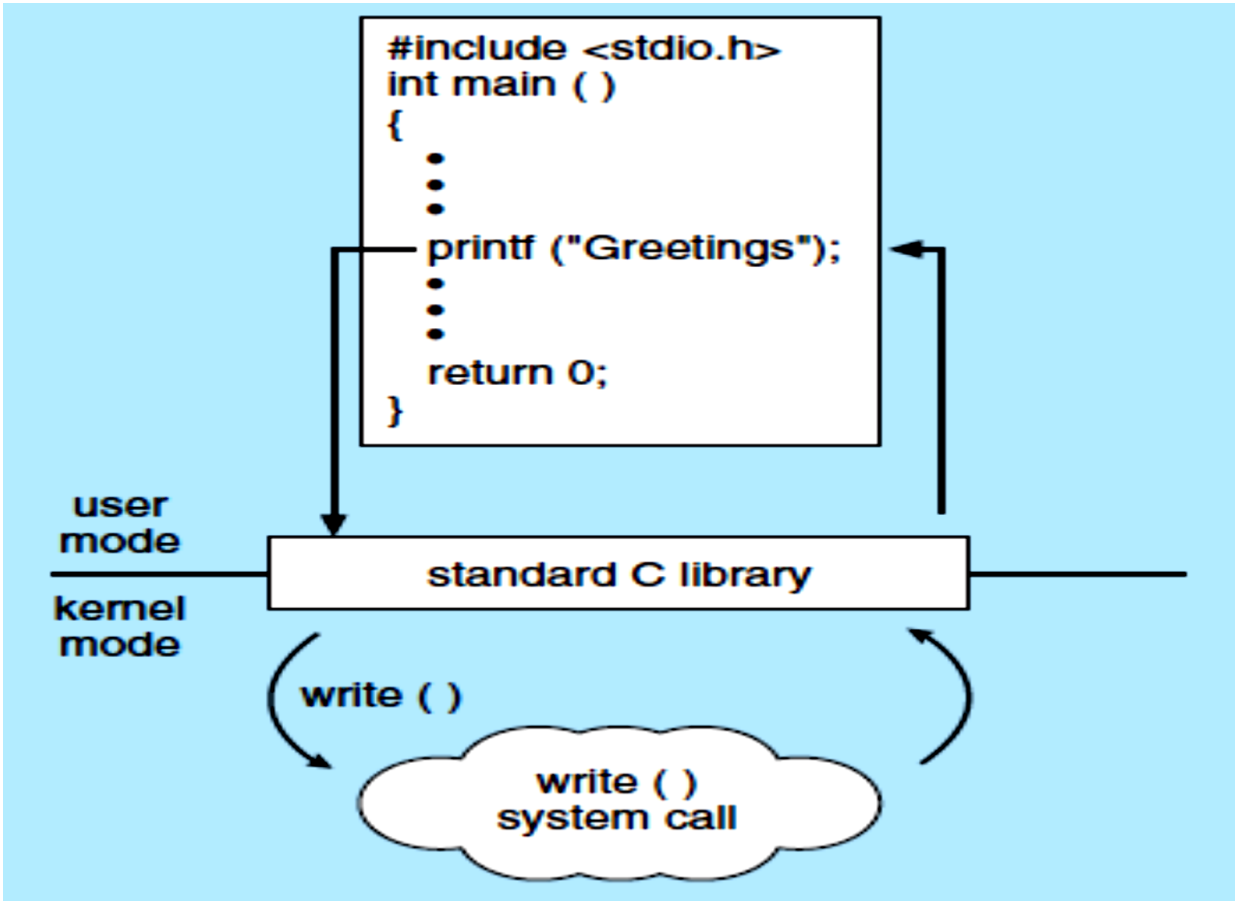
- A system call is a mechanism that provides the interface between program and OS.
- System call offers the services of OS to the user programs via API (Application Programming Interface). Three most common APIs are:
  1) Win32 API for Windows
  2) Java API for the Java virtual machine (JVM)
  3) POSIX API for POSIX-based systems (UNIX, Linux, and Mac OS X)
- A program requests a service from OS where each service has its own system call and each system call has a number associated with it.
- This number is sent to the kernel so that OS knows what the requested system call is.
  - System-call interface maintains a table indexed according to these numbers in the kernel code.
  - The system call interface invokes the intended system call in OS kernel



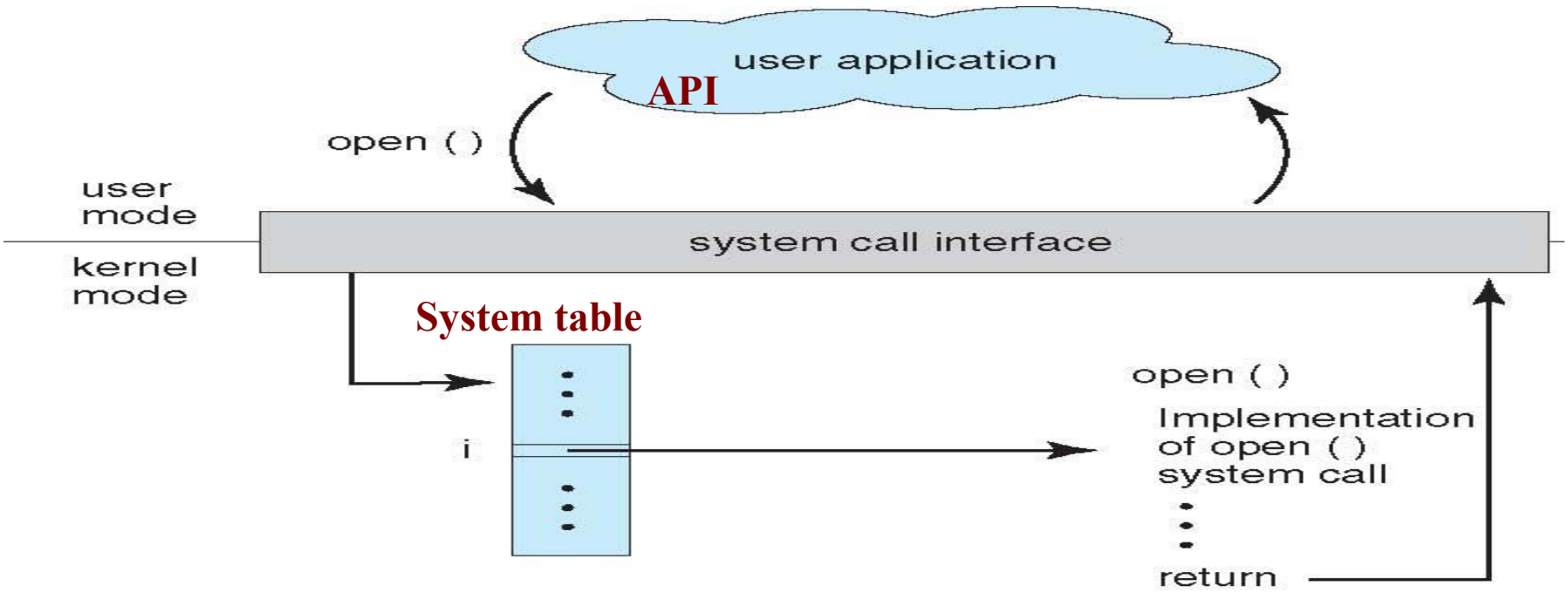
Common system calls in Linux OS are: open, read, write, close, wait, exec, fork, exit, kill ……

Standard C Library Example

- C program invoking printf() library call, which calls write() system call.
- The C library takes the value returned by write() and passes it back to the user program.



System Calls



For example, when the open() command is executed, the transition will take place from the user mode to the system kernel mode, and therefore the system call must be used, and this is done by calling the number associated with the system call that is found through the system table.

Why do you need system calls in Operating System?

There are various situations where you must require system calls in the OS.

Following of the situations are as follows:

1) When a file system wants to create or delete a file.
2) Network connections require the system calls to sending and receiving data packets.
3) If you want to read or write a file, you need to system calls.
4) If you want to access hardware devices, including a printer, scanner, you need a system call.
5) System calls are used to create and manage new processes.

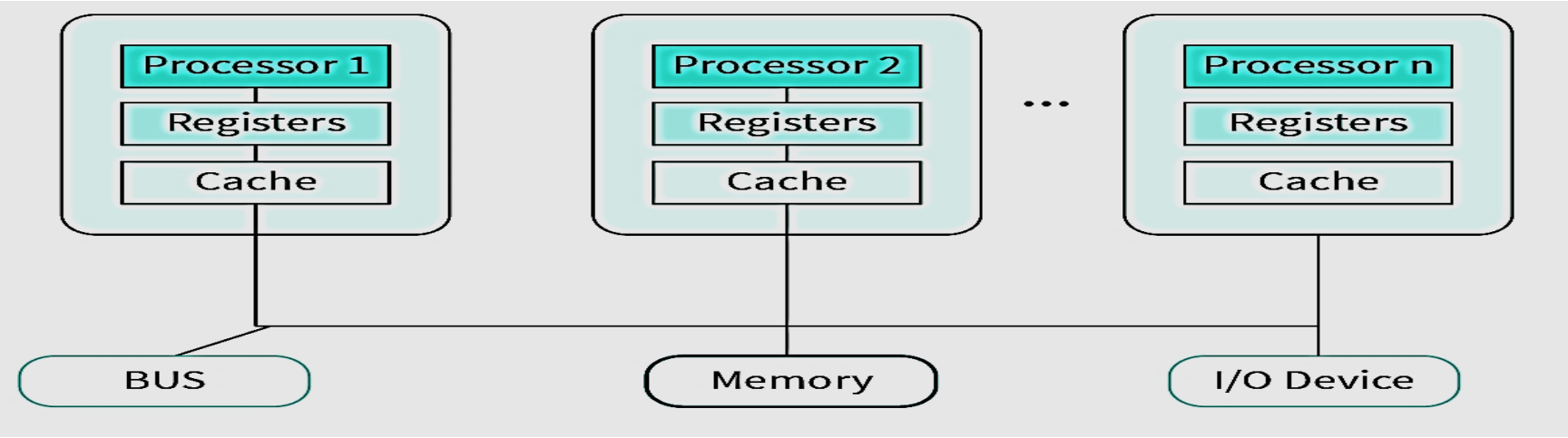
Examples of Windows and Unix System Calls

Process	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	Fork() Exit() Wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	Open() Read() Write() Close()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	Ioctl() Read() Write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	Getpid() Alarm() Sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	Pipe() Shmget() Mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorgroup()	Chmod() Umask() Chown()

Characteristics of Modern Operating Systems

Multiprocessor Symmetric Multiprocessors (SMP)

- There are multiple processors → OS must handle Processor scheduling
- These processors share same main memory, I/O device and bus
- All processors can perform the same functions (hence the term symmetric).



Advantages of Symmetric Multiprocessors (SMP)

1) **Increased throughput** (number of processes that are completed per time)
   - ✓ Several processes or threads can run in parallel across all of the processors at the same time. This will increase the throughput of the system.

2) **Economical**: no need to make copies of data and distribute it among several processors.

   Because they can share peripherals, mass storage, and power supplies.
   - ✓ If several programs operate on the same set of data, it is cheaper to store those data on one disk and to have all the processors share them than to have many computers with local disks and many copies of the data.

3) **Performance**: divide the work on all processors.



Interleaving (Multiprogramming, one processor)



Interleaving and overlapping (Multiprocessing, two processors)



Multiprogramming and Multiprocessing

- In a single-processor multiprogramming system, processes are interleaved in time to yield the appearance of simultaneous execution
  - Only one process can execute at a time; meanwhile, all other processes are waiting for the processor (actual parallel processing is not achieved)
- In a multiprocessor system, actual parallel processing is achieved.
  - it is possible not only to interleave (alternating between them mutually) the execution of multiple processes, but also to overlap (intersect/interconnect/extend) them.
  - More than one process can be running simultaneously, each on a different processor.

Advantages of multiprocessor systems (parallel system)

4) **Increased reliability (Availability/fault – tolerant):**

   In symmetric multiprocessor, because all processors can perform the same functions, the failure of a single processor does not halt the system. Instead, the system can continue to function at reduced performance.

For example, if we have 10 processors working together on a job and one processor failed, then the remaining 9 processors must pick up a share of the work of the failed processor. Thus, the entire system is still working but slower by 10%.

Characteristics of Modern Operating Systems

Multicore (chip multiprocessor)

- A multicore is a single CPU or processor with two or more independent processing units called cores on a single chip.
- Typically, each core consists of all components of an independent processor (such as ALU, registers and control unit) in addition to L1, L2 and shared L3 cache
- A single processor in a multicore read and executing program instructions on separate cores at same time, increasing the overall speed of the system's program execution.
- They work in such a way that the computer system appears to have several processors, although they are cores, not processors.
- A CPU with two cores is called a dual-core processor while a processor with four cores is called a quad-core processor, six ("hexa-core"), and eight ("octa-core")

The main advantage of a multicore

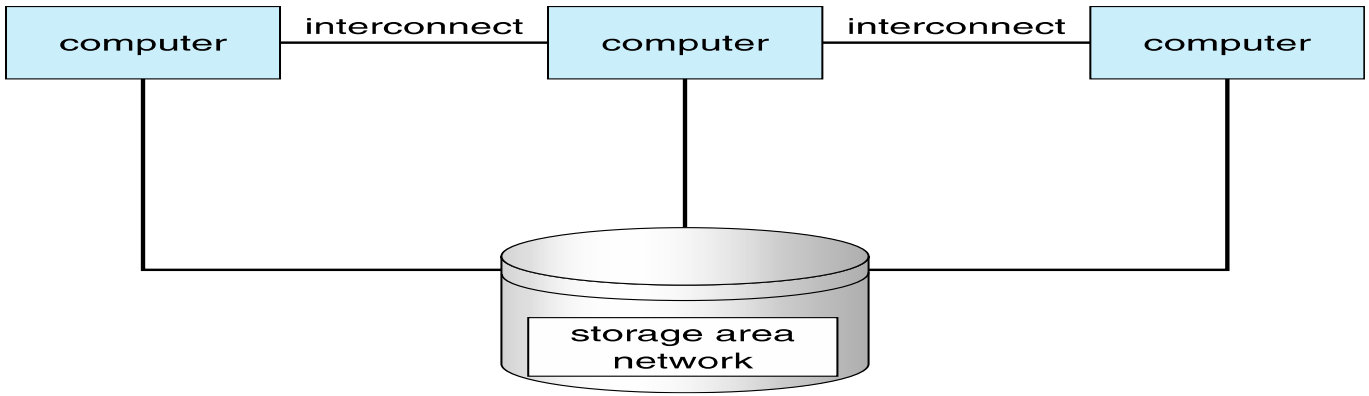
1) Executing multiple instructions simultaneously on separate cores.
2) It increases the overall speed and also implements parallel computing.
3) Multicore more efficient than multiple chips with single cores Because:
   - ✓ On-chip communication is faster than between-chip communication.
   - ✓ In addition, one chip with multiple cores uses significantly less power than multiple single-core chips.

- Multicore is used for applications such as embedded systems, networking, Digital Signal Processing (DSP) and graphics processing units (GPU).

Latest computers have multiple processor each with multiple cores to read and execute several instructions at a time.

Characteristics of Modern Operating Systems
Multicore (chip multiprocessor)
Clustered Systems

- Like multiprocessor systems, but multiple systems working together
- A clustered system uses multiple CPUs to complete a task.
- The clustered computers share storage and are closely linked via LAN networking.
- Usually sharing storage via a storage-area network (SAN)
- It is different from multiprocessor in that clustered system consists of two or more individual systems tied together.



The clustered system can be of the following forms:

1) Asymmetric clustering

- In this form, one machine is in hot-standby mode while the other is running the applications.
- The hot-standby host machine does nothing but monitor the active server. If that server fails, the hot-standby host becomes the active server.

2) Symmetric clustering

- In this form, two or more machines run the applications and monitor each other at the same time.
- This mode is more efficient because it uses all available machines.
- It can be used only if multiple applications are available to be executed

Advantages of Clustered Systems

1) **Provides a high-availability service which survives failures:**

- **A layer of cluster software runs on cluster nodes.**
- **Each node can monitor one or more nodes over the LAN.**
- **If the monitored machine fails, the monitoring machine can take ownership of its storage and restart the applications that were running on the failed machine.**

2) Provide high-performance computing environments:

- Since a cluster consists of several computer systems connected via a network, clusters can also be used to provide high-performance computing environments.

➢ because they can run an application concurrently on all computers in the cluster.

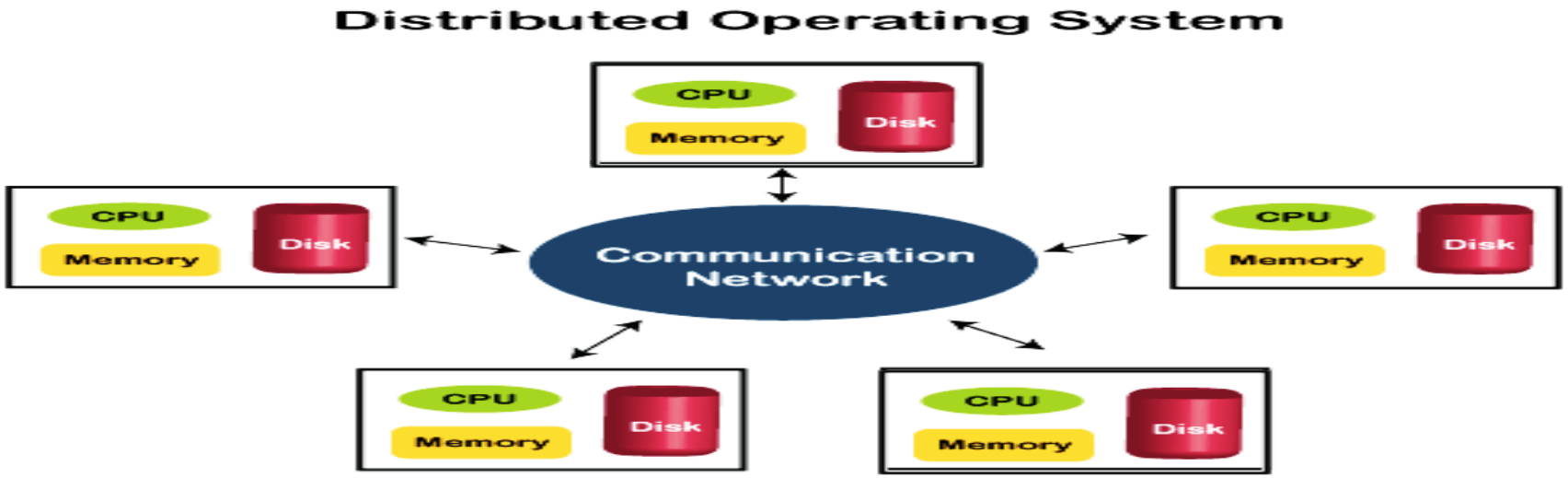
✓ The application must have been written specifically to take advantage of the cluster, however. This involves a technique known as parallelization, which divides a program into separate components that run in parallel on individual cores in a computer or computers in a cluster.

✓ Typically, these applications are designed so that once each computing node in the cluster has solved its portion of the problem, the results from all the nodes are combined into a final solution.

Characteristics of Modern Operating Systems

Distributed operating systems

- It is a collection of autonomous/independent/separated computer systems (Nodes) (possibly heterogenous) but interconnected with each other using a shared communication network allowing them to share resources
- Each of these nodes contains a small part of the distributed OS software.
- OS running on the different computers acts like a single OS
- Provides the illusion of a single main memory and single secondary memory
- The "**distributed**" nature of the system is hidden from the processes
- OS manages all the "distributed" aspects



Characteristics of Modern Operating Systems

Distributed operating systems

- **Distributed OS is not installed on a single machine, it is divided into parts, and these parts are loaded on different machines.**
- **A part of the Distributed OS is installed on each machine to make their communication possible.**

Advantages Distributed operating systems

1) User can access the files or software which are not actually present on his system but on some other system connected within this network.

2) Achieves reliability and availability (If one system fails, the other systems can continue to operate)

3) Process A communicates with process B the same way whether they are on same computer or not.

4) Processes can be moved among different computers

5) Used for distributed file system

6) Mobile offloading (mobile capabilities less than of task requirements to run e.g. rendering task can be on server)

Characteristics of Modern Operating Systems

Object-oriented design

- Used for adding modular extensions to a small kernel
- Enables programmers to customize an operating system without disrupting integrity


Any Questions ?
Thank you